EFREI Paris

Projet Deep Learning

Système d'aide au tri radiologique

Classification supervisée, détection d'anomalies, multimodalité et traçabilité expérimentale

Auteur(s)	Mouradi Iliasse et Mahi Karima
Groupe	14
Encadrant	Philip Tchatchoua
Établissement	EFREI Paris
Année universitaire	2025–2026

26 mars 2026

Résumé

Ce projet vise à concevoir un système d'aide au tri radiologique capable d'analyser des radiographies thoraciques, de proposer des scores de pathologies, d'identifier des cas atypiques via reconstruction, d'exploiter une modalité textuelle lorsqu'un compte-rendu est disponible, puis d'exposer le tout dans un prototype applicatif testable localement. Le pipeline a été implémenté en PyTorch avec suivi obligatoire sous MLflow et interface Streamlit. La partie image principale repose sur ChestMNIST, tandis que la composante multimodale s'appuie sur IU X-Ray / OpenI pour la variante *image + texte* et sur NIH Chest X-rays pour une variante complémentaire *image + métadonnées structurées*. Ce document décrit les données, les prétraitements, les choix de modélisation, les métriques, les outils utilisés et l'état des résultats disponibles au moment de l'export.

Section 1

*

Table des matières

1 * ii

2 Problème 1

2.1 Contexte opérationnel 1

2.2 Questions de recherche 1

2.3 Hypothèses et périmètre 2

3 Données 3

3.1 ChestMNIST pour la classification supervisée et l'anomalie 3

3.2 IU X-Ray / OpenI pour la composante image + texte 3

3.3 NIH Chest X-rays comme variante complémentaire 4

3.4 Comparaison synthétique des jeux de données 4

3.5 Contraintes d'accès, de format et de gouvernance 5

3.6 Pertinence et limites par composante 5

3.7 Limites des données 5

4 Analyse exploratoire des données 6

4.1 Lecture qualitative de l'EDA 8

4.2 Premiers constats sur le texte radiologique 8

5 Préparation des données 10

5.1 Prétraitement des images 10

5.2 Préparation des labels 10

5.3 Préparation du texte 10

5.4 Stratégie anti-fuite et reproductibilité 11

5.5 Gestion du déséquilibre 11

5.6 Synthèse des pipelines de préparation 12

5.7 Methodologie de conduite de projet 12

5.8 Demarche methodologique generale 12

5.9 Outils et environnement expérimental 13

5.10 Contraintes de calcul et choix de compromis 14

6 Modélisation supervisée 15

6.1 CNN simple entraîné depuis zéro 15

6.2 CNN pré-entraîné avec transfer learning 15
6.3 Vision Transformer compact 15
6.4 Choix d'optimisation 16
6.5 Critères de comparaison 16

7 Détection d'anomalies par AE / VAE 17
7.1 Pourquoi un autoencodeur dans ce projet 17
7.2 Protocole d'entraînement 17
7.3 Interprétation du score d'anomalie 18
7.4 Points forts et limites 18

8 Modélisation multimodale image + compte-rendu 19
8.1 Représentation image 19
8.2 Représentation texte 19
8.3 Stratégie de fusion 19
8.4 Organisation des expériences multimodales 19
8.5 Comparaisons prévues 20
8.6 Robustesse à l'absence d'une modalité 20
8.7 Variante NIH et discussion méthodologique 20

9 Évaluation 21
9.1 Pourquoi ces métriques 21
9.2 Temps d'entraînement et matrices adaptées 21
9.3 Comparaison finale des runs supervisés 23
9.4 Lecture finale de la brique d'anomalie 24
9.5 Lecture finale des runs multimodaux 25
9.6 Snapshot final complet 26
9.7 Analyse par classe et implications 26

10 Tracking MLflow et traçabilité 27
10.1 Organisation pratique des runs 27
10.2 Artefacts suivis 28
10.3 Traçabilité jusqu'au déploiement 28

11 Démonstrateur applicatif 30
11.1 Architecture logicielle de l'application 30
11.2 Scénario d'usage 30
11.3 Limites applicatives et perspectives 32

12 Analyse critique **33**

12.1 Lecture d'ensemble des résultats 33

12.2 Robustesse et généralisation 33

12.3 Biais, limites et précautions d'interprétation 34

12.4 Intérêt pédagogique et scientifique du projet 34

12.5 Enjeux éthiques et usage responsable 34

13 Conclusion et perspectives **35**

14 * **36**

Section 2

Problème

Le tri radiologique consiste à aider un praticien ou un service d'imagerie à prioriser rapidement les examens potentiellement pathologiques, ambigus ou atypiques. Dans le cas des radiographies thoraciques, cette aide doit rester prudente : l'objectif n'est pas de remplacer le radiologue, mais d'accélérer le repérage des cas urgents, de soutenir la lecture et d'offrir un support de démonstration technologique.

Le sujet imposait une vision système et non une simple tâche de classification. Le projet devait combiner :

— une classification supervisée multi-label sur radiographies thoraciques ;
— une brique de détection d'anomalies par autoencodeur ;
— une preuve de concept multimodale image + texte ;
— un suivi expérimental rigoureux avec MLflow ;
— un démonstrateur interactif cohérent avec les modèles entraînés.

Du point de vue IA, la difficulté principale vient du caractère multi-label des clichés thoraciques, du déséquilibre marqué entre pathologies fréquentes et rares, de la variabilité d'apparence des anomalies, et de l'ambiguïté introduite par des modalités hétérogènes comme le texte ou les métadonnées.

2.1 Contexte opérationnel

Dans un service de radiologie, le tri ne se résume pas à une simple prédiction binaire. Il s'inscrit dans un flux complet incluant l'acquisition de l'image, la transmission PACS, la lecture spécialisée, l'éventuelle validation clinique et la priorisation des cas urgents. Un système d'aide au tri pertinent doit donc produire une information exploitable rapidement, tout en restant compréhensible et suffisamment robuste pour ne pas dégrader l'organisation du service.

Dans le cas thoracique, les besoins sont très variés. Certaines pathologies comme l'épanchement pleural, le pneumothorax ou les infiltrats pulmonaires peuvent justifier une attention plus rapide. D'autres signes sont plus subtils, rares ou d'interprétation difficile. En conséquence, un outil utile doit pouvoir couvrir plusieurs cas de figure : donner des probabilités multi-label, signaler les cas atypiques pour relecture prioritaire, et tirer profit d'un texte lorsqu'il est disponible.

Le projet a donc été pensé comme un compromis entre réalisme clinique, faisabilité académique et reproductibilité informatique. Le but n'est pas d'atteindre un niveau de validation clinique certifiable, mais de proposer une architecture de système cohérente et traçable qui montre comment articuler plusieurs briques d'apprentissage profond autour d'un même usage.

2.2 Questions de recherche

Le rapport s'organise autour de quatre questions principales :

— dans quelle mesure un pipeline supervisé simple mais rigoureux peut-il déjà fournir une aide pertinente au tri sur radiographies thoraciques ;
— l'erreur de reconstruction d'un autoencodeur permet-elle de distinguer de façon utile les

images normales des cas atypiques ou pathologiques ;
— l'ajout d'une modalité textuelle améliore-t-il la qualité de la prédiction par rapport aux branches image seule et texte seul ;
— comment garantir que les expériences, les artefacts et le démonstrateur restent parfaitement cohérents entre eux.

2.3 Hypothèses et périmètre

Le périmètre choisi reste volontairement délimité. Le projet se concentre sur des radiographies thoraciques 2D, sur des jeux de données publics, et sur une implémentation reproductible sur une machine de travail personnelle disposant d'un GPU milieu de gamme. Plusieurs hypothèses simplificatrices ont été adoptées.

Premièrement, pour la classification supervisée, les labels fournis par ChestMNIST sont utilisés comme référence opératoire, malgré le bruit potentiel hérité du dataset d'origine. Deuxièmement, pour la détection d'anomalies, la notion de "normalité" est assimilée à l'absence de label positif dans ChestMNIST. Troisièmement, pour la multimodalité, la variante principale s'appuie sur de vrais comptes-rendus, tandis que la variante NIH est considérée comme un complément orienté *métadonnées/annotations* plutôt que comme une vraie source narrative.

Ces choix réduisent la complexité du problème sans l'appauvrir artificiellement. Ils permettent surtout d'obtenir un pipeline cohérent, réexécutable, compatible avec MLflow et suffisamment lisible pour être discuté dans un cadre universitaire.

Section 3

Données

3.1 ChestMNIST pour la classification supervisée et l'anomalie

Le jeu de données principal est **ChestMNIST**, sous-ensemble du benchmark MedMNIST v2 [1]. La documentation officielle du benchmark est disponible sur `https://medmnist.com/` et le dépôt Git utilisé dans le projet est `https://github.com/MedMNIST/MedMNIST`. Ce sous-ensemble reprend des radiographies thoraciques annotées pour 14 pathologies, avec des splits officiels `train/val/test`. Dans le projet, ces splits sont utilisés tels quels via le package `medmnist`, ce qui limite le risque de fuite de données.

Le téléchargement retenu est celui recommandé par l'API officielle du projet MedMNIST. En pratique, l'environnement Python installe directement la bibliothèque depuis le dépôt Git officiel via `pip install -upgrade git+https://github.com/MedMNIST/MedMNIST.git`, puis les données sont récupérées à l'aide de l'appel `ChestMNIST(split="test", download=True, size=224)` ou de variantes de taille `64`, `128` et `224`. Cette procédure est reprise dans le script du projet `scripts/download_medmnist_official.py`, qui génère en outre un résumé local des tailles et des splits téléchargés.

Les tailles observées localement sont :

— train : 78 468 images ;
— validation : 11 219 images ;
— test : 22 433 images.

ChestMNIST est particulièrement adapté à un cadre pédagogique : il permet d'expérimenter plusieurs résolutions, des modèles de complexité différente et un protocole reproductible, tout en conservant un lien direct avec les pathologies thoraciques issues de ChestX-ray8 [2].

3.2 IU X-Ray / OpenI pour la composante image + texte

Pour satisfaire explicitement la contrainte *image + compte-rendu*, la branche multimodale principale utilise **IU X-Ray / OpenI** [3]. Le jeu préparé localement contient 5 910 entrées, réparties en 4 138 exemples d'entraînement, 592 de validation et 1 180 de test.

Chaque ligne associe :

— une image thoracique ;
— un champ textuel `report_text` ;
— 6 labels thoraciques (`atelectasis`, `cardiomegaly`, `effusion`, `edema`, `pneumothorax`, `hernia`).

Un point méthodologique important doit être signalé : ces 6 labels ont été dérivés automatiquement à partir du compte-rendu par règles lexicales avec gestion simple de la négation. Ils constituent donc des *weak labels*. Cette décision est acceptable pour une preuve de concept académique, mais impose une lecture prudente des résultats multimodaux, car la modalité texte peut contenir une information très proche de la cible.

Ce choix répond directement à la formulation la plus exigeante du sujet, à savoir l'intégration d'une composante *image + compte-rendu*. Autrement dit, même sans soutenance orale, le dossier

écrit reste cohérent : la conformité à l'exigence textuelle est assurée par IU X-Ray, tandis que NIH est conservé comme variante complémentaire.

3.3 NIH Chest X-rays comme variante complémentaire

Le sujet mentionnait également **NIH Chest X-rays**. Une fiche descriptive du jeu de données utilisée dans le cadre du projet est disponible sur https://www.innovatiana.com/fr/datasets/nih-chest-x-rays. Pour rester cohérent avec cette consigne, une seconde branche de données a été préparée à partir du corpus NIH ChestX-ray8 [2]. Elle contient 112 120 images, 30 805 patients uniques, et une répartition patient-level en 73 499 exemples de train, 13 025 de validation et 25 596 de test.

L'acquisition du corpus NIH a été automatisée via `kagglehub`. La procédure minimale reprise dans le projet est la suivante : `import kagglehub` puis `kagglehub.dataset_download("nih-chest-xrays/dat` Cette logique est encapsulée dans le script `scripts/import_nih_kagglehub.py`, qui téléchage le dataset, reconstruit les chemins image, applique le split patient-level et produit le fichier `nih_multimodal_metadata.csv` ainsi qu'un résumé d'import.

Dans cette variante, la seconde modalité n'est pas un compte-rendu libre mais un texte structuré `metadata_text` construit à partir de l'âge, du sexe, de la vue, du suivi et des annotations de boîtes englobantes. Cette piste est utile pour discuter la notion de multimodalité pratique, mais la comparaison principale du rapport reste centrée sur la variante *image + texte* avec IU X-Ray.

La lecture finale retenue dans ce manuscrit est donc la suivante : **ChestMNIST** constitue le socle principal pour la classification supervisée et l'anomalie, **IU X-Ray / OpenI** satisfait la contrainte *image + texte*, et **NIH Chest X-rays** documente une extension complémentaire *image + métadonnées*. Cette formulation rend le projet auto-suffisant sur le plan documentaire, y compris en l'absence de présentation orale.

3.4 Comparaison synthétique des jeux de données

Les trois sources retenues n'ont pas exactement le même rôle dans le projet. Le tableau 1 résume leur positionnement.

Table 1 – Comparaison des jeux de données utilisés dans le projet.

Dataset	Taille utilisée	Modalités	Rôle dans le projet	Intérêt principal
ChestMNIST	78 468 train / 11 219 val / 22 433 test	Image + labels	classification supervisée et anomalie	benchmark léger, splits officiels, 14 labels thoraciques
IU X-Ray / OpenI	4 138 train / 592 val / 1 180 test	Image + texte libre + weak labels	branche multimodale principale	vraie preuve de concept image + compte-rendu
NIH Chest X-rays	73 499 train / 13 025 val / 25 596 test	Image + métadonnées/annotations	variante multimodale complémentaire	forte volumétrie, structuration patient, richesse d'annotations

Cette comparaison montre que les datasets ne sont pas redondants. ChestMNIST facilite l'en-

traînement rapide et propre de modèles de référence. IU X-Ray permet de satisfaire la dimension *image + texte*, indispensable pour la fusion multimodale. NIH, enfin, permet de relier le projet à un jeu de données thoraciques de grande ampleur, plus proche d'une exploitation en situation semi-réaliste, même si la deuxième modalité y est plus structurée que narrative.

3.5 Contraintes d'accès, de format et de gouvernance

Les contraintes pratiques d'accès aux données ont eu un impact direct sur le pipeline. ChestMNIST est le plus simple à intégrer : le package officiel `medmnist`, installé depuis le dépôt Git `MedMNIST/MedMNIST`, gère automatiquement le téléchargement, le parsing et les splits. IU X-Ray nécessite en revanche une préparation plus explicite : récupération des JSONL, extraction des images, nettoyage des textes et construction du CSV multimodal. NIH est encore différent, car le dataset est volumineux, distribué via `kagglehub` dans le projet, et nécessite ensuite une reconstitution des chemins image ainsi qu'un split patient-level pour éviter des fuites entre train et validation.

Du point de vue du rapport, il est important de souligner que ces jeux de données sont publics et utilisables dans un cadre académique, mais qu'ils ne doivent pas être confondus avec des données cliniques validées pour une utilisation médicale en production. Leur intérêt est scientifique et pédagogique. Les biais de sélection, le bruit d'annotation et les simplifications opérées lors du prétraitement impliquent qu'aucune conclusion clinique forte ne doit être tirée au-delà du cadre du projet.

3.6 Pertinence et limites par composante

L'utilisation de ChestMNIST pour les modèles supervisés est justifiée par sa simplicité d'usage, sa reproductibilité et sa compatibilité avec plusieurs tailles d'image. Son principal inconvénient est de compresser des radiographies thoraciques en un format très simplifié, ce qui peut lisser des motifs diagnostiques fins.

IU X-Ray est beaucoup plus adapté à la dimension sémantique du projet, car il fournit un texte libre plus proche d'un compte-rendu réaliste. En revanche, sa taille est nettement plus modeste, ce qui rend les estimations plus sensibles à la variance et au déséquilibre.

NIH est utile pour discuter la multimodalité au sens large, notamment lorsque des métadonnées structurées ou des annotations de localisation sont disponibles. Toutefois, comme le sujet insistait aussi sur l'exploitation de comptes-rendus, NIH ne pouvait pas suffire seul à remplir la partie *image + texte* au sens strict.

3.7 Limites des données

Trois limites majeures ressortent dès cette phase :

- un fort déséquilibre des classes, notamment pour les pathologies rares ;
- du bruit d'annotation hérité des datasets sources ;
- une possible fuite conceptuelle dans IU X-Ray lorsque les labels sont extraits du texte lui-même.

Section 4

Analyse exploratoire des données

L'EDA a été générée automatiquement pour ChestMNIST à l'aide du script `scripts/generate_eda.py`. Les figures principales sont reproduites ci-dessous.

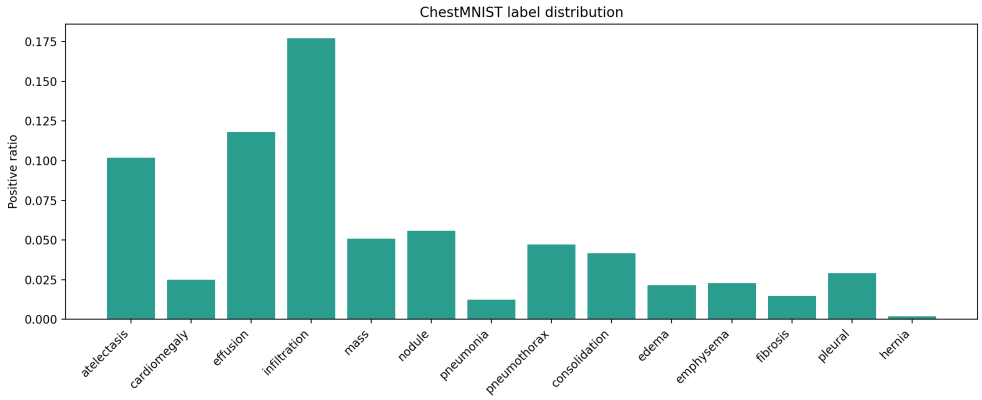

Figure 1 – Distribution des classes positives dans le split d'entraînement ChestMNIST.

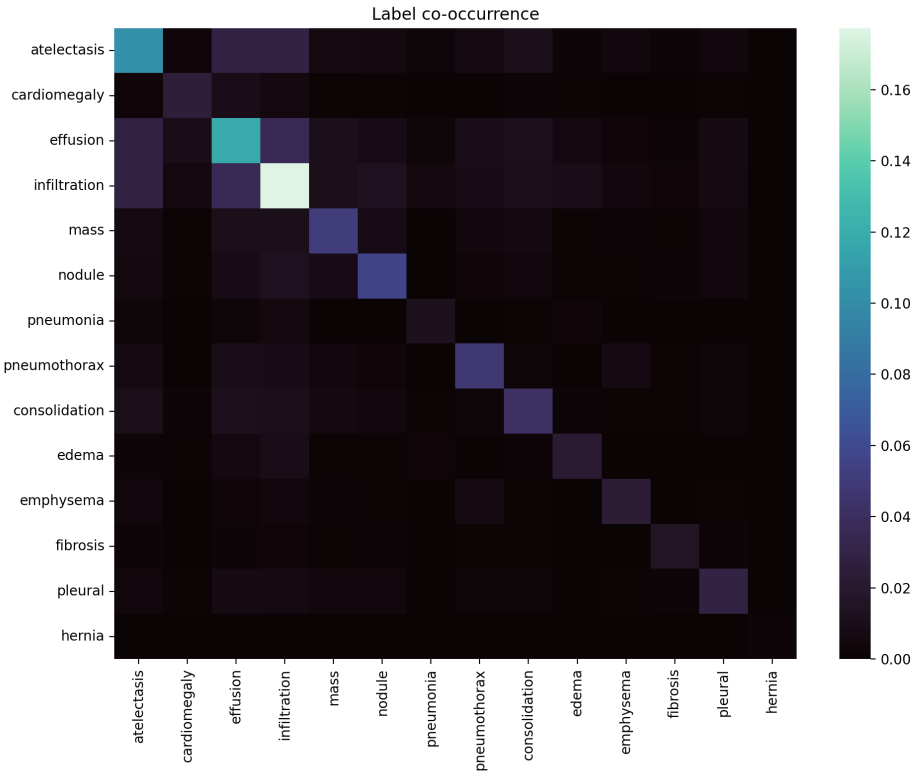

Figure 2 – Co-occurrences de labels dans ChestMNIST.

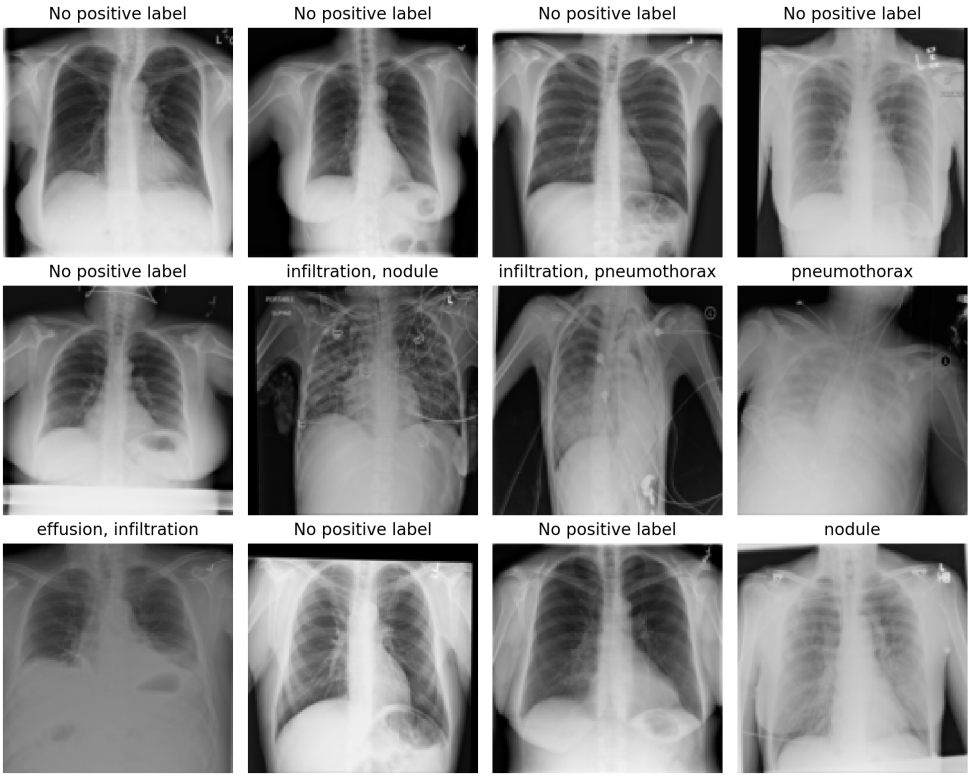

Figure 3 – Exemples de radiographies du split d'entraînement.

L'analyse exploratoire montre d'abord un fort déséquilibre. Dans le split d'entraînement, les pathologies les plus fréquentes sont `infiltration` (17.73%), `effusion` (11.80%) et `atelectasis` (10.19%). À l'inverse, `hernia` (0.18%), `pneumonia` (1.25%) et `fibrosis` (1.48%) sont rares. Environ 54.0% des images du train ne portent aucun label positif, ce qui est particulièrement utile pour l'entraînement de l'autoencodeur sur des cas supposés normaux.

Les co-occurrences les plus fortes concernent `effusion` + `infiltration`, `atelectasis` + `infiltration` et `atelectasis` + `effusion`. Ce point confirme que l'on est bien dans un contexte multi-label où plusieurs atteintes peuvent être présentes simultanément.

Du côté textuel, IU X-Ray présente des comptes-rendus courts à moyens, avec une moyenne d'environ 31 tokens, une médiane à 29 et un 90^{e}percentile à 50. Les labels dérivés y sont eux aussi déséquilibrés : `effusion` et `pneumothorax` sont fréquents, alors que `hernia` et `edema` sont peu représentés.

Table 2 – Distribution des labels dans le split d'entraînement ChestMNIST.

Label	Positifs	Ratio	Commentaire
atelectasis	7 996	10.19%	classe fréquente, souvent co-occurrente avec effusion et infiltration
cardiomegaly	1 950	2.49%	support limité mais cliniquement important
effusion	9 261	11.80%	l'une des pathologies les plus stables du corpus
infiltration	13 914	17.73%	classe la plus fréquente du jeu d'entraînement
mass	3 988	5.08%	intermédiaire, potentiellement hétérogène
nodule	4 375	5.58%	classe sensible à la résolution et au bruit
pneumonia	978	1.25%	rare, donc difficile en termes de rappel
pneumothorax	3 705	4.72%	cliniquement pertinent pour le tri
consolidation	3 263	4.16%	fréquence moyenne
edema	1 690	2.15%	faible support, mais souvent discriminable
emphysema	1 799	2.29%	classe peu fréquente
fibrosis	1 158	1.48%	rare et potentiellement difficile
pleural	2 279	2.90%	support limité
hernia	144	0.18%	extrêmement rare

4.1 Lecture qualitative de l'EDA

L'EDA confirme que le problème n'est pas seulement un problème de reconnaissance visuelle. Il s'agit aussi d'un problème de calibration statistique. Dans un jeu fortement déséquilibré, un modèle peut apprendre à bien ranger les exemples par ordre de risque sans pour autant bien détecter les classes rares au seuil de décision standard. C'est exactement la raison pour laquelle le rapport privilégie le ROC-AUC et l'Average Precision à l'accuracy brute.

Les exemples visuels révèlent par ailleurs que plusieurs clichés sont peu contrastés, très homogènes ou marqués par des différences d'acquisition. Ce constat justifie le choix d'augmentations légères mais non agressives. Une augmentation trop forte pourrait artificiellement dégrader des motifs thoraciques déjà fins dans les versions redimensionnées du benchmark.

Les co-occurrences les plus fortes ne doivent pas être interprétées comme des relations causales. Elles signalent plutôt que certains tableaux radiologiques ou certaines conventions d'annotation apparaissent ensemble plus souvent. D'un point de vue modélisation, cela signifie qu'un classifieur multi-label peut exploiter des corrélations utiles, mais aussi qu'il risque d'apprendre des raccourcis entre classes au lieu de repérer chaque signe visuel de manière indépendante.

4.2 Premiers constats sur le texte radiologique

Le texte d'IU X-Ray contient majoritairement des formulations courtes, standardisées, avec une forte répétition lexicale autour des motifs radiologiques classiques. Cette propriété simplifie la vectorisation par vocabulaire et GRU, mais elle augmente aussi le risque que certaines ex-

pressions deviennent presque directement prédictives des labels dérivés. Ce point sera discuté plus loin dans l'analyse critique, car il influence fortement l'interprétation des performances du modèle texte seul et du modèle fusionné.

Du côté NIH, le texte structuré `metadata_text` n'est pas un vrai langage narratif. Il agit plutôt comme une modalité tabulaire serialisée sous forme de chaîne. Son intérêt est donc différent : il ne sert pas à tester une compréhension sémantique riche, mais à vérifier qu'un pipeline multimodal peut exploiter proprement des informations non visuelles disponibles au moment du tri.

Section 5

Préparation des données

5.1 Prétraitement des images

Pour la classification supervisée, les images ChestMNIST sont redimensionnées à la taille attendue par chaque modèle. Une normalisation de type ImageNet est utilisée pour les modèles RGB. Des augmentations légères sont activées en entraînement : retournement horizontal aléatoire et rotation faible. Pour l'autoencodeur, la normalisation est désactivée afin de conserver une reconstruction directement interprétable dans l'espace image.

Ce choix de prétraitement est volontairement pragmatique. Dans un projet académique, il est tentant de multiplier les transformations, les normalisations spécifiques et les recettes de *data augmentation*. Ici, l'objectif a été de conserver un pipeline sobre, explicable et compatible avec plusieurs familles de modèles. Les transformations retenues servent donc surtout à homogénéiser les tailles d'entrée, limiter le surapprentissage et conserver une cohérence entre entraînement, évaluation et démonstrateur.

La normalisation ImageNet pour les entrées RGB peut sembler discutable dans un contexte médical. Elle a cependant deux avantages pratiques : elle est directement compatible avec les backbones torchvision pré-entraînés et elle stabilise les statistiques d'entrée pour les branches convolutionnelles. Pour l'autoencodeur, cette normalisation aurait compliqué l'interprétation de la reconstruction. C'est pourquoi la branche anomalie conserve des images non normalisées au sens ImageNet.

5.2 Préparation des labels

La tâche supervisée est multi-label. Chaque sortie est donc traitée par une perte `BCEWithLogitsLoss`. Afin de compenser partiellement le déséquilibre, des `pos_weight` sont calculés sur le train et intégrés à la fonction de perte lorsque la configuration l'autorise.

Pour l'anomalie, une cible binaire est définie par la présence d'au moins une pathologie positive. Les images normales du split train servent à apprendre la distribution de reconstruction, tandis que validation et test contiennent à la fois des cas normaux et anormaux.

Cette construction est importante, car elle sépare clairement l'objectif de reconstruction de l'objectif de classification. L'autoencodeur n'apprend pas à discriminer explicitement les maladies. Il apprend à reconstruire des exemples supposés normaux. Le score d'anomalie découle ensuite d'une déviation par rapport à cette distribution. Il s'agit donc d'un usage *indirect* des labels, limité à la définition de la normalité pendant la constitution du train.

5.3 Préparation du texte

Le texte multimodal est tokenisé par une règle simple à base d'expressions régulières, puis converti en indices via un vocabulaire construit exclusivement sur le split d'entraînement. Les séquences sont tronquées/paddées à une longueur maximale de 128 tokens, avec une taille de vocabulaire plafonnée à 5 000 mots. Ce choix reste volontairement simple et reproductible.

Le recours à une tokenisation simple a été préféré à un tokenizer biomédical lourd pour trois

raisons. D'abord, cela rend le pipeline totalement autonome et facilement exécutable hors ligne. Ensuite, cela simplifie fortement la sauvegarde et le rechargement dans l'application Streamlit. Enfin, dans un contexte de comptes-rendus relativement courts et fortement standardisés, une modélisation lexicale simple constitue un point de départ raisonnable avant d'envisager des encodeurs plus coûteux.

5.4 Stratégie anti-fuite et reproductibilité

Le pipeline respecte plusieurs garde-fous :

— splits officiels `train/val/test` pour ChestMNIST ;
— split au niveau patient pour la variante NIH ;
— vocabulaire textuel construit sur le train uniquement ;
— calibrage du seuil d'anomalie sur la validation normale uniquement ;
— seed fixe `42` avec configuration déterministe PyTorch/CUDA ;
— sauvegarde systématique du meilleur checkpoint de validation.

L'anti-fuite mérite une attention particulière dans un projet médical. Une fuite ne provient pas seulement d'une mauvaise séparation train/test. Elle peut aussi apparaître lorsqu'un vocabulaire textuel est construit sur l'ensemble du corpus, lorsqu'une calibration de seuil utilise des données de test, ou lorsqu'un même patient peut se retrouver dans plusieurs splits. Les scripts du projet ont donc été structurés pour que chaque étape de préparation soit indexée sur le split courant et ne regarde pas au-delà de ce qui lui est permis.

5.5 Gestion du déséquilibre

Le déséquilibre des labels thoraciques est un problème central du projet. Les classes les plus rares auraient été rapidement écrasées par les classes fréquentes si la perte n'avait pas été pondérée. La solution retenue consiste à calculer un `pos_weight` par classe à partir du split d'entraînement. Ce choix n'annule pas complètement le déséquilibre, mais il oblige l'optimisation à ne pas ignorer les classes rares.

Ce choix est plus sobre qu'une stratégie de sur-*sampling* ou qu'une perte focalisée. Il présente l'avantage de rester facile à tracer, à expliquer et à reproduire. Il s'accorde aussi avec l'objectif du projet, qui est de montrer un pipeline rigoureux avant de chercher des optimisations de pointe.

5.6 Synthèse des pipelines de préparation

Table 3 – Synthèse des prétraitements appliqués selon la composante du projet.

Composante	Taille image	Normalisation	Augmentation	Particularité
Supervision SimpleCNN	128	ImageNet	flip horizontal + rotation	apprentissage depuis zéro
Supervision ResNet18 transfer	224	ImageNet	flip horizontal + rotation	transfer learning backbone gelé
Supervision ResNet18 FT partiel	224	ImageNet	flip horizontal + rotation	`layer4 + fc` dégelés
Supervision TinyViT	128	ImageNet	flip horizontal + rotation	patches + encodeur Transformer compact
Autoencodeur	64	aucune	légère augmentation train	reconstruction en espace image
Multimodal IU X-Ray	128	ImageNet	flip horizontal train	texte vectorisé, max length 128

5.7 Methodologie de conduite de projet

Au-delà de la méthodologie d'apprentissage, l'organisation du travail a suivi une logique de pilotage par **tableau Kanban**. Cette approche a été retenue pour structurer le projet en tâches courtes, visibles et réparties entre les grandes briques imposées par le sujet : données, EDA, modélisation supervisée, anomalie, multimodalité, tracking MLflow, application et rapport.

Le tableau Kanban a permis de distinguer explicitement plusieurs états de travail, par exemple :

— *à faire* pour les tâches identifiées mais non commencées ;
— *en cours* pour les composants en développement ou en entraînement ;
— *à valider* pour les tâches techniquement terminées mais encore en attente de vérification ;
— *terminé* pour les livrables stabilisés et intégrés au dépôt.

Cette méthode présente un intérêt particulier dans un projet de deep learning, car le temps de travail n'est pas uniquement du temps de codage. Une partie importante du projet est consommée par le téléchargement et la préparation des jeux de données, les essais de configurations, les temps d'entraînement, l'analyse des artefacts MLflow et la rédaction du rapport. Le tableau Kanban a donc servi à visualiser les dépendances entre tâches et à maintenir une progression cohérente entre expérimentation, validation technique et production des livrables.

Dans le rapport, ce point méthodologique est important car il explique la manière dont le projet a été conduit : non pas comme une suite informelle d'essais, mais comme un enchaînement organisé de tâches pilotées, priorisées et validées au fur et à mesure de l'avancement.

5.8 Demarche methodologique generale

La méthodologie suivie dans le projet peut être résumée comme une démarche itérative et reproductible articulée autour de six étapes. La première a consisté à cadrer le sujet et à le découper en briques fonctionnelles cohérentes : classification supervisée, anomalie, multimodalité, tracking expérimental, application et rapport. Cette phase de cadrage a été directement reliée au tableau Kanban, qui a servi de support de planification et de priorisation.

La deuxième étape a concerné l'acquisition et la structuration des données. Elle a consisté

à sélectionner 'ChestMNIST' comme base principale pour la partie image, 'IU X-Ray / OpenI' pour la composante *image + texte*, puis 'NIH Chest X-rays' comme variante complémentaire. Les scripts d'import ont été conçus pour produire des formats exploitables directement par les loaders PyTorch du projet, avec une séparation claire entre fichiers bruts, CSV préparés et résumés d'import.

La troisième étape a été l'analyse exploratoire. Son objectif n'était pas seulement descriptif, mais décisionnel. L'EDA a permis d'identifier le déséquilibre des classes, de visualiser les co-occurrences de labels, d'observer le type d'images disponibles et d'évaluer la structure du texte radiologique. Ces observations ont ensuite guidé les choix de prétraitement, de métriques et de pondération des classes.

La quatrième étape a été la mise en place des protocoles d'entraînement. Pour la supervision, l'objectif a été de comparer trois familles de modèles correspondant explicitement aux exigences du sujet. Pour la détection d'anomalies, la méthodologie a reposé sur l'apprentissage d'une distribution normale puis sur la calibration d'un seuil de reconstruction. Pour la multimodalité, la démarche a consisté à comparer des baselines unimodales à un modèle de fusion, afin d'isoler l'apport réel de la seconde modalité.

La cinquième étape a porté sur l'évaluation et la traçabilité. Les expériences ont été enregistrées dans MLflow avec leurs hyperparamètres, leurs métriques, leurs artefacts visuels et leurs checkpoints. Cette phase a permis de conserver une continuité entre entraînement, comparaison de runs, sélection des modèles et intégration dans l'application.

Enfin, la sixième étape a concerné l'industrialisation académique du projet, c'est-à-dire la production d'un démonstrateur exploitable et d'un rapport argumenté. La logique suivie a donc été la suivante : partir des données, construire des pipelines reproductibles, comparer les approches, tracer les résultats, puis transformer ces résultats en livrables cohérents et présentables.

5.9 Outils et environnement expérimental

Le projet repose principalement sur Python 3.11, PyTorch 2.11, Torchvision, MedMNIST, pandas, scikit-learn, MLflow et Streamlit. L'exécution finale a été relancée sur une machine Windows 11 dotée d'un processeur Intel Core i7-10750H, de 16 Go de RAM et d'un GPU NVIDIA GeForce RTX 2070 Max-Q avec CUDA 12.8.

Le matériel utilisé fait partie intégrante de la méthodologie du projet, car il conditionne directement les résolutions retenues, la taille des batches, le nombre d'époques, le choix d'un *TinyViT* compact plutôt qu'un Transformer plus lourd, ainsi que la durée totale d'exécution de la pipeline finale. L'ensemble des entraînements finaux, du suivi MLflow, des tests applicatifs et de la compilation du rapport a été réalisé dans cet environnement local.

Table 4 – Configuration matérielle et logicielle utilisée pour le projet.

Elément	Valeur
OS	Windows 11 Famille (build 10.0.26200)
CPU	Intel Core i7-10750H, 6 coeurs / 12 threads
RAM	16 Go
GPU	NVIDIA GeForce RTX 2070 with Max-Q Design
VRAM	environ 8 Go
Python	3.11.9
PyTorch	2.11.0+cu128
Torchvision	0.26.0+cu128
Framework d'app	Streamlit 1.53.1
Tracking	MLflow 3.10.1

5.10 Contraintes de calcul et choix de compromis

Le matériel disponible a influencé plusieurs choix d'architecture et d'hyperparamètres. Un GPU de type RTX 2070 Max-Q permet de lancer l'ensemble du projet, mais il ne rend pas pour autant tous les choix calculatoires gratuits. En particulier, un ViT standard pré-entraîné à grande résolution aurait imposé un coût mémoire et temporel beaucoup plus important. C'est l'une des raisons qui ont conduit à l'introduction d'un *TinyViT* maison, plus modeste mais plus compatible avec un rendu de projet complet.

De même, le nombre d'époques, les tailles de batch, la résolution des images et l'usage soit du gel du backbone soit d'un dégel partiel dans ResNet18 ont été pensés comme des compromis entre qualité d'apprentissage, durée d'exécution et stabilité globale. Dans un cadre industriel, ces compromis seraient probablement rediscutés sur des serveurs multi-GPU ou sur des benchmarks plus lourds. Dans un cadre académique, ils permettent de conserver un pipeline final réaliste et exécutable de bout en bout.

Section 6

Modélisation supervisée

Trois familles de modèles sont comparées pour la partie supervisée.

Le choix de ces trois familles répond directement à la consigne du sujet, mais il suit aussi une logique expérimentale. Le **SimpleCNN** sert de baseline contrôlée. **ResNet18** sert de point de comparaison en transfer learning avec un backbone largement standardisé, d'abord en mode backbone gelé puis en fine-tuning partiel plus ambitieux. **TinyViT** sert enfin à représenter le paradigme Transformer sans rendre le coût d'entraînement prohibitif. Ensemble, ces trois familles permettent de discuter non seulement les performances, mais aussi la sensibilité au calcul, la facilité de convergence et la cohérence avec les contraintes du projet.

6.1 CNN simple entraîné depuis zéro

Le premier modèle est un CNN compact conçu pour l'apprentissage à partir de zéro. Il empile quatre blocs convolution + batch normalization + ReLU, suivis d'un *adaptive average pooling* et d'une couche linéaire finale. Cette architecture sert de base de comparaison franche : elle est plus légère, plus rapide à entraîner et plus facile à interpréter qu'un backbone pré-entraîné.

Ce modèle est particulièrement intéressant pour un projet pédagogique, car il permet de mesurer ce que le pipeline apprend *vraiment* sans héritage fort issu d'un pré-entraînement externe. Si les performances sont déjà raisonnables, cela signifie que la chaîne de données, la fonction de perte, les métriques et la gestion des splits sont saines. En ce sens, le CNN simple joue aussi le rôle de garde-fou expérimental.

6.2 CNN pré-entraîné avec transfer learning

Le deuxième modèle est un ResNet18 [4] pré-entraîné sur ImageNet. Deux régimes ont été évalués dans le projet. Le premier conserve le backbone gelé et n'optimise que la tête de classification, ce qui fournit une base de transfer learning rapide et stable. Le second dégèle seulement `layer4` et `fc`, avec un learning rate plus faible pour le backbone que pour la tête. Cette seconde variante, exécutée en résolution 224, constitue finalement le meilleur run supervisé du projet.

Le gel complet du backbone réduit le nombre de paramètres effectivement optimisés, accélère l'entraînement et diminue le risque d'instabilité dans un contexte où la taille d'image et la taille de lot restent modérées. En contrepartie, il limite l'adaptation fine du modèle au domaine radiologique. Le dégel partiel de `layer4 + fc` offre un compromis plus convaincant : l'essentiel du backbone reste stable, mais les dernières représentations peuvent mieux s'aligner sur la radiologie thoracique sans exploser le coût d'entraînement.

6.3 Vision Transformer compact

Le troisième modèle est un petit transformer visuel inspiré de ViT [5]. Il ne s'agit pas d'un ViT industriel pré-entraîné, mais d'une implémentation compacte adaptée aux ressources disponibles : extraction de patches par convolution, ajout d'un jeton `cls`, embeddings positionnels appris, encodeur Transformer puis tête linéaire. L'intérêt de cette architecture est de couvrir explicitement la famille Transformer demandée par le sujet.

Cette architecture ne doit pas être lue comme une tentative de battre tous les modèles convolutionnels sur ce benchmark. Elle est plutôt conçue comme une exploration du paradigme Transformer dans un cadre matériel raisonnable. Le rapport met donc autant l'accent sur la justification du choix que sur le chiffre brut final.

6.4 Choix d'optimisation

Les entraînements supervisés utilisent AdamW, un scheduler cosinus, un `early stopping` sur la validation et une BCE multi-label pondérée. Les tailles d'images et de batch diffèrent selon les modèles afin de conserver un coût calculatoire raisonnable.

Le protocole de comparaison cherche à être homogène sans être artificiellement identique. Il aurait été possible d'imposer la même taille d'image et le même batch à tous les modèles, mais cela aurait pénalisé inutilement certaines architectures. Le parti pris retenu consiste donc à conserver une même philosophie d'optimisation tout en adaptant la résolution et le batch size à la famille de modèle.

Table 5 – Configurations principales des modèles supervisés.

Modèle	Taille image	Batch size	Epoques max	Learning rate	Particularité
SimpleCNN	128	64	10	1e-3	apprentissage depuis zéro
ResNet18 transfer	224	32	5	3e-4	backbone pré-entraîné gelé
ResNet18 partial FT 224	224	24	12	2e-4 / 2e-5	`layer4 + fc`, learning rates différen
TinyViT	128	32	6	3e-4	encodeur Transformer compact

6.5 Critères de comparaison

Les trois familles de modèles ne sont pas comparées uniquement sur la qualité du ROC-AUC final. Quatre dimensions sont considérées :

— la qualité de classement multi-label sur le test ;
— la stabilité de la validation au cours des époques ;
— le rapport entre coût de calcul et gain obtenu ;
— la facilité de réutilisation dans le démonstrateur applicatif.

Ce dernier point est important. Un modèle excellent mais trop lourd à charger ou trop lent à inférer dans l'application perd une partie de son intérêt dans un projet orienté démonstration. La dimension système oblige donc à dépasser la seule logique "leaderboard".

Section 7

Détection d'anomalies par AE / VAE

La brique d'anomalie repose sur un autoencodeur convolutionnel simple. L'encodeur applique trois convolutions stridées successives et compresse l'image vers un espace latent de 128 canaux. Le décodeur reconstruit ensuite l'image par convolutions transposées.

La perte retenue est une `L1Loss` entre image d'origine et reconstruction. Le score d'anomalie est la moyenne des erreurs absolues de reconstruction par pixel. L'intuition est directe : si le modèle a surtout vu des cas normaux, il doit moins bien reconstruire les cas pathologiques, atypiques ou hors distribution.

Le seuil de décision est calibré sur la validation à partir du 95^{e}percentile des scores des cas normaux. Ce choix simple a l'avantage d'être traçable, facilement reproductible et compatible avec une interface de démonstration.

7.1 Pourquoi un autoencodeur dans ce projet

La brique d'anomalie a un rôle différent de la classification supervisée. Elle n'a pas vocation à dire *quelle* pathologie est présente, mais à signaler qu'une image s'écarte de ce que le modèle considère comme habituel. Dans un scénario de tri, cette information complémente utilement les probabilités supervisées. Une image peut obtenir une prédiction modérée sur plusieurs classes tout en présentant un score d'anomalie élevé, ce qui peut justifier une relecture prioritaire.

Le choix d'un autoencodeur plutôt que d'un VAE est lié à la lisibilité du projet. L'AE convolutionnel est plus direct à implémenter, plus facile à expliquer dans un rapport de master ou d'ingénieur, et suffisamment expressif pour démontrer le principe d'une détection par reconstruction. Un VAE aurait ajouté une couche de complexité probabiliste utile, mais non indispensable pour la preuve de concept visée ici.

7.2 Protocole d'entraînement

L'entrée du modèle est une radiographie redimensionnée en 64×64. Le train ne contient que des images sans label positif, de sorte que le modèle apprenne prioritairement une distribution "normale". La validation et le test contiennent à la fois des cas normaux et anormaux, ce qui permet de mesurer la qualité discriminante du score de reconstruction.

Le protocole suivi est le suivant :

- apprentissage sur le split train normal uniquement ;
- calcul des scores d'erreur absolue moyenne sur le split val ;
- définition d'un seuil à partir du quantile 95% des cas normaux de validation ;
- application du même seuil au split test pour obtenir F1, précision, rappel et accuracy ;
- journalisation des reconstructions et de l'histogramme des scores dans MLflow.

Cette chaîne est simple, mais elle possède une vraie logique expérimentale. Le seuil n'est pas choisi *a posteriori* sur le test ; il est calibré avant toute évaluation finale. Cette distinction est essentielle pour maintenir une interprétation correcte des résultats.

7.3 Interprétation du score d'anomalie

Le score calculé ne mesure pas directement une distance clinique à une pathologie. Il mesure une difficulté de reconstruction dans l'espace image. Une erreur élevée peut provenir d'une lésion, mais aussi d'un artefact d'acquisition, d'un cadrage inhabituel, d'un contraste atypique ou d'un motif anatomique peu fréquent dans les cas normaux du train. En pratique, cela veut dire qu'un score d'anomalie doit être interprété comme un signal de vigilance, pas comme un diagnostic.

Cette propriété est en réalité cohérente avec le cahier des charges du sujet. Le projet doit *identifier des cas atypiques, ambigus, rares ou hors distribution.* La catégorie visée est donc plus large que la maladie au sens strict. Un bon module d'anomalie peut avoir de la valeur même s'il ne distingue pas parfaitement toutes les pathologies.

7.4 Points forts et limites

L'approche par autoencodeur présente plusieurs avantages. Elle est simple, visuelle, explicable, et produit des artefacts d'analyse qualitatifs très utiles dans un rapport : grilles de reconstruction, histogrammes de scores, inspection des faux positifs et faux négatifs. Elle est aussi indépendante de la tâche de classification proprement dite, ce qui apporte une forme de redondance utile au système.

Ses limites sont toutefois importantes. D'une part, la notion de normalité apprise dépend fortement du train disponible. D'autre part, la reconstruction peut rester de bonne qualité sur certains cas pathologiques répétitifs, et inversement se dégrader sur des variations non pathologiques. Enfin, l'évaluation binaire "normal vs au moins un label positif" simplifie fortement la réalité clinique. Cette brique doit donc être comprise comme un module d'alerte exploratoire, pas comme un outil autonome de décision.

Section 8

Modélisation multimodale image + compte-rendu

8.1 Représentation image

La voie image multimodale utilise un petit encodeur convolutionnel produisant un embedding dense de taille 128. Cette branche est volontairement légère afin que la comparaison *image seule / texte seul / fusion* mette davantage en évidence l'apport de la modalité textuelle que l'effet d'un backbone très profond.

L'intérêt de cette simplicité est aussi méthodologique. Si la branche image était trop puissante, la fusion pourrait ne presque rien apporter, simplement parce que l'image seule dominerait le problème. A l'inverse, une branche image trop faible rendrait la fusion artificiellement dépendante du texte. Le petit encodeur convolutionnel constitue donc un compromis raisonnable pour comparer les trois variantes de manière plus lisible.

8.2 Représentation texte

La voie texte repose sur une couche d'embedding, suivie d'un GRU bidirectionnel et d'un pooling masqué par `attention_mask`. Cette approche reste plus simple qu'un encodeur de type BERT, mais elle est parfaitement suffisante pour une preuve de concept reproductible.

Le GRU bidirectionnel présente plusieurs avantages dans ce contexte. Il capture l'ordre local des mots mieux qu'un simple sac de mots, reste plus léger qu'un transformeur de langage, et se recharge facilement dans l'application. Sur des comptes-rendus courts, il peut suffire à représenter l'information sémantique essentielle sans faire exploser le temps d'entraînement.

8.3 Stratégie de fusion

La fusion retenue est une **fusion intermédiaire** : les représentations image et texte sont encodées séparément, concaténées, puis injectées dans une petite tête MLP. En entraînement, un *modality dropout* peut annuler aléatoirement une partie de l'information image ou texte, afin de tester une forme minimale de robustesse à l'absence d'une modalité.

Ce choix de fusion a été retenu pour des raisons à la fois conceptuelles et pratiques. Une fusion précoce aurait imposé d'harmoniser très tôt des données de nature radicalement différente. Une fusion tardive, elle, aurait davantage rapproché le problème d'un assemblage de scores. La fusion intermédiaire permet au contraire de laisser chaque modalité construire une représentation adaptée avant interaction, ce qui est bien aligné avec un projet de preuve de concept.

8.4 Organisation des expériences multimodales

Trois runs distincts sont prévus sur la branche IU X-Ray :

- `image_only` : l'image est la seule modalité active ;
- `text_only` : seule la séquence textuelle est utilisée ;
- `fusion` : les deux représentations sont concaténées avant prédiction.

Cette structure est fondamentale pour l'interprétation. Un score de fusion n'a de sens que s'il

est comparé à des branches unimodales entraînées dans un cadre cohérent. Sans ces baselines, il serait impossible de savoir si la multimodalité aide vraiment ou si une seule modalité portait déjà presque toute l'information utile.

8.5 Comparaisons prévues

Conformément au sujet, trois variantes sont implémentées :

— un modèle *image only* ;
— un modèle *text only* ;
— un modèle *fusion image + texte*.

Cette comparaison doit toutefois être interprétée avec prudence sur IU X-Ray, car les labels sont extraits du compte-rendu. Si le texte domine trop fortement, cela peut traduire moins un gain multimodal qu'un raccourci sémantique lié au weak labeling.

8.6 Robustesse à l'absence d'une modalité

Le code du projet inclut un test supplémentaire sur le modèle de fusion : l'évaluation avec texte désactivé et l'évaluation avec image désactivée. Cette expérience est intéressante à double titre. D'une part, elle renseigne sur la dépendance réelle du modèle à chaque modalité. D'autre part, elle rapproche le pipeline d'un usage applicatif réaliste dans lequel le texte peut être absent, partiel ou de mauvaise qualité.

Dans une lecture saine des résultats, un modèle de fusion n'est pas nécessairement meilleur parce qu'il obtient le plus haut score absolu. Il peut aussi être meilleur parce qu'il se dégrade plus gracieusement lorsqu'une modalité manque, ou parce qu'il reste plus stable qu'une branche unimodale face à certains cas ambigus.

8.7 Variante NIH et discussion méthodologique

La variante NIH n'est pas le coeur de la comparaison finale, mais elle est utile pour discuter un point souvent oublié dans les projets multimodaux : en pratique, la "modalité texte" n'est pas toujours un compte-rendu libre. Elle peut être un ensemble de métadonnées patient, d'informations de contexte ou d'annotations semi-structurées. Construire `metadata_text` sous forme séquentielle permet de réutiliser la même infrastructure de chargement, de vocabulaire et de modèle texte sans créer un pipeline totalement séparé.

Cette décision a un intérêt pédagogique fort. Elle montre que la multimodalité du projet n'est pas limitée à un cas unique, mais qu'elle repose sur une abstraction logicielle suffisamment générale pour accepter plusieurs types de deuxième modalité.

Section 9

Évaluation

Dans un cadre multi-label et fortement déséquilibré, l'accuracy seule est insuffisante. Les métriques retenues dans le code sont :

— ROC-AUC macro et micro ;
— Average Precision macro et micro ;
— F1 macro et micro ;
— accuracy par sous-ensemble, utilisée uniquement comme indicateur secondaire ;
— analyse par classe à partir des courbes, barplots et matrices de confusion adaptées.

9.1 Pourquoi ces métriques

Le choix des métriques n'est pas un détail de présentation. Il conditionne la manière dont on juge un système de tri. Le ROC-AUC évalue la capacité du modèle à classer les positifs avant les négatifs, indépendamment d'un seuil fixe. L'Average Precision est plus sensible au déséquilibre et donc particulièrement informative sur des classes rares. Le F1, lui, introduit explicitement un seuil de décision et reflète donc davantage l'usage opérationnel.

Dans un projet de tri radiologique, la lecture croisée de ces métriques est essentielle. Un modèle peut avoir un bon ROC-AUC mais un F1 faible si son seuil par défaut ne correspond pas bien à la distribution des probabilités. Inversement, une classe extrêmement rare peut obtenir un ROC-AUC élevé tout en restant peu exploitable à cause d'une précision très faible.

Table 6 – Rôle des métriques utilisées dans le projet.

Métrique	Ce qu'elle mesure	Intérêt dans le projet
ROC-AUC macro	moyenne des ROC-AUC par classe	vue globale équilibrée entre classes rares et fréquentes
ROC-AUC micro	ROC-AUC sur les sorties aplaties	vue globale dominée par les classes fréquentes
AP macro	moyenne des précisions-rappels par classe	très informative sur les labels rares et déséquilibrés
F1 macro	moyenne des F1 par classe	reflète mieux l'usage au seuil fixe pour toutes les classes
Subset accuracy	exactitude de la combinaison complète de labels	indicateur secondaire très exigeant en multi-label

9.2 Temps d'entraînement et matrices adaptées

Les temps d'entraînement sont extraits directement des timestamps de début et de fin enregistrés dans MLflow. Ils sont reportés dans les tableaux de synthèse afin de relier explicitement performance et coût de calcul. Cette information est importante dans un projet académique, car elle permet de justifier les compromis de résolution, de batch size et de complexité modèle au regard du matériel disponible.

En complément des courbes ROC-AUC et des barplots par classe, le projet produit aussi des **matrices adaptées** au type de tâche. Pour les problèmes multilabel, une grille de matrices

de confusion 2x2 est générée classe par classe. Pour la détection d'anomalies, une matrice de confusion binaire est produite au seuil calibré sur la validation. Ces figures sont journalisées dans MLflow et sauvegardées dans les artefacts finaux.

Instantane MLflow. Les tableaux ci-dessous ont ete generes automatiquement depuis le suivi MLflow le 26/03/2026 12 :04. Tous les runs finaux attendus sont disponibles dans ce snapshot. Les quickstarts ne sont pas reportes ici afin de ne conserver que les experiences finales du projet.

Table 7 – Synthese des runs supervises finaux.

Modele	Statut	Device	Epoques	Duree	Val.	Test ROC-AUC	Test AP / F1
SimpleCNN	termine	cuda	10	17m 34s	0.7443	0.7402	0.1217 / 0.1577
ResNet18 transfer	termine	cuda	5	18m 00s	0.7118	0.7058	0.1131 / 0.1466
ResNet18 partial FT 224	termine	cuda	12	42m 29s	0.7920	0.7940	0.1844 / 0.1907
TinyViT	termine	cuda	6	6m 24s	0.5559	0.5427	0.0596 / 0.0814

Table 8 – Synthese du run final de detection d'anomalies.

Modele	Statut	Device	Epoques	Duree	Val.	Test ROC-AUC	Test AP / F1
Conv Autoencoder	termine	cuda	12	4m 44s	0.4666	0.4800	0.4534 / 0.0840

Table 9 – Synthese des runs multimodaux finaux.

Modele	Statut	Device	Epoques	Duree	Val.	Test ROC-AUC	Test AP / F1
Image only	termine	cuda	8	5m 47s	0.5799	0.5663	0.1463 / 0.1726
Text only	termine	cuda	8	4m 32s	0.9986	0.9953	0.9823 / 0.9445
Fusion image + texte	termine	cuda	10	6m 45s	0.9974	0.9967	0.9881 / 0.9583

Table 10 – Robustesse du modele de fusion quand une modalite manque.

Modele	ROC-AUC sans texte	ROC-AUC sans image
Fusion IU X-Ray	0.5480	0.9966

Table 11 – Traçabilite des checkpoints deployes dans le demonstrateur.

Composante	Checkpoint deploye	Experience MLflow	Run name	Run ID
Supervision	`supervised/resnet18_partial_finetune_224`	`chest_finetune_224`	`resnet18_partial_finetune_224`	`d88d549209ef4b1aab7de8a84e40b8ae`
Anomalie	`anomaly/conv_autoencoder`	`chestmnist_anomaly`	`conv_autoencoder`	`59d650266dbb46d18e93eaf63963e47e`
Multimodal	`multimodal/iu_xray_fusion`	`radiology_multimodal`	`iu_xray_fusion_text`	`3a933d1dcf864d7c8c1dd5db6e9a9313`

Le manifest de deploiement associe au demonstrateur est exporte dans `../artifacts/deployment_manifest`

9.3 Comparaison finale des runs supervisés

Les runs supervisés finaux sont désormais disponibles, avec une variante supplémentaire de **ResNet18** en fine-tuning partiel sur des images de taille 224. Sur ChestMNIST, le meilleur compromis obtenu dans le cadre du projet est maintenant celui de **ResNet18 partial FT 224**, avec un ROC-AUC macro test de 0.7940, une Average Precision macro de 0.1844 et un F1 macro de 0.1907 au seuil global 0.5. Ce run dépasse nettement le **SimpleCNN** historique (ROC-AUC macro 0.7402) ainsi que la variante **ResNet18 transfer** à backbone gelé (0.7058). **TinyViT** reste très en retrait avec un ROC-AUC macro de 0.5427.

Ce nouveau classement est instructif. Il montre qu'un backbone pré-entraîné n'exprime pas tout son potentiel lorsqu'il reste entièrement gelé, mais qu'un **dégel partiel ciblé** des couches hautes peut apporter un gain très significatif sans basculer dans un fine-tuning complet coûteux. Le **SimpleCNN** conserve une valeur méthodologique comme baseline robuste et simple, mais ce n'est plus lui qui offre la meilleure qualité de classement.

L'analyse par classe du meilleur run supervisé reste instructive. Certaines classes rares présentent un ROC-AUC très élevé, comme `edema`, `emphysema` ou `hernia`, mais cela ne se traduit pas automatiquement par un bon F1 ni par une bonne Average Precision lorsque le support reste faible. À l'inverse, `effusion` et `atelectasis` combinent des scores plus homogènes, ce qui traduit une meilleure stabilité prédictive sur des classes mieux représentées.

Le couple `ROC-AUC macro = 0.7940` et `F1 macro = 0.1907` illustre bien la difficulté du problème. Le classement est nettement meilleur qu'auparavant, mais le seuil uniforme à 0.5 reste trop brutal pour transformer ce gain en décisions positives optimales sur toutes les classes. La calibration par classe confirme ce point : sur le meilleur **ResNet18 partial FT 224**, le F1 macro test passe de 0.1907 à 0.2431 après calibration, et la subset accuracy de 0.1685 à 0.3785.

Une autre lecture intéressante concerne l'écart entre macro et micro. Pour le meilleur **ResNet18 partial FT 224**, le ROC-AUC micro atteint 0.8158, au-dessus du macro, ce qui signale une meilleure exploitation des classes fréquentes. En parallèle, l'Average Precision macro monte nettement à 0.1844, ce qui confirme que le gain ne se limite pas à un simple effet de ranking global mais reflète aussi une meilleure qualité par classe.

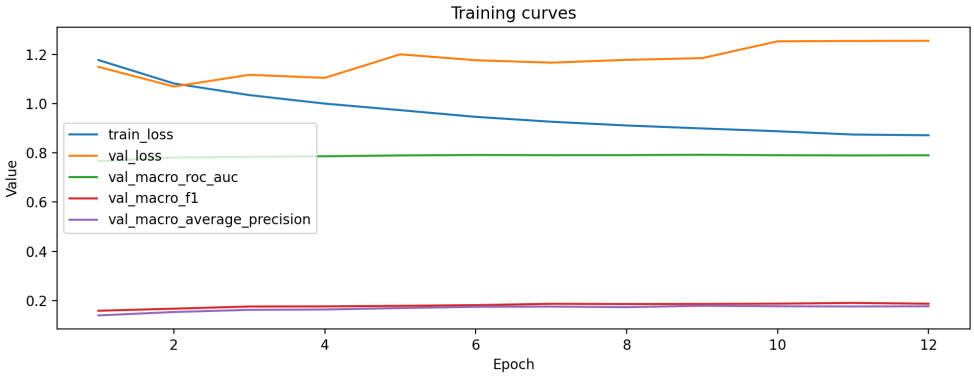

Figure 4 – Courbes d'entraînement du meilleur run supervisé ResNet18 partial FT 224.

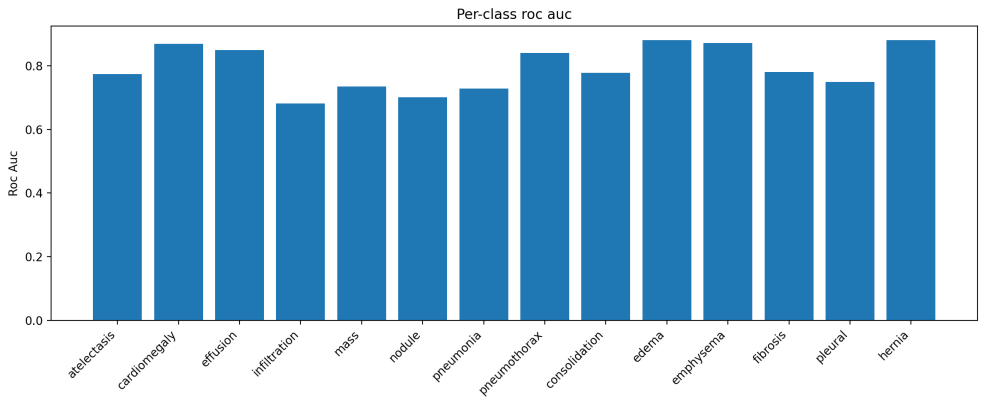

Figure 5 – ROC-AUC par classe pour le meilleur run supervisé ResNet18 partial FT 224.

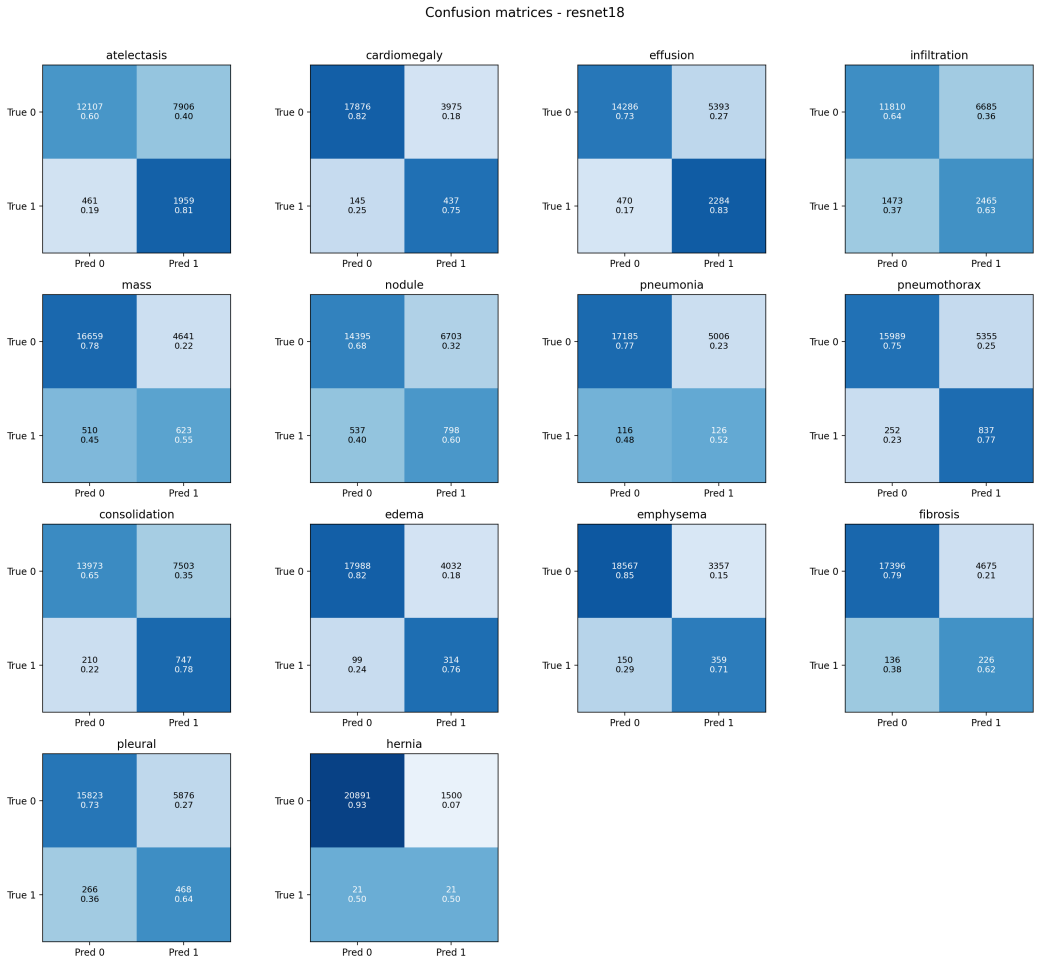

Figure 6 – Matrices de confusion par classe pour le meilleur run supervisé ResNet18 partial FT 224.

9.4 Lecture finale de la brique d'anomalie

Le run final de **Conv Autoencoder** atteint un ROC-AUC test de 0.4800, une Average Precision de 0.4534 et un F1 de 0.0840. Sous cette définition de l'anomalie, la séparation entre cas normaux et cas pathologiques n'est donc pas satisfaisante. La métrique de classement est même légèrement inférieure au hasard, ce qui indique que l'erreur de reconstruction ne constitue

pas ici un proxy robuste de la présence d'une pathologie.

Ce résultat n'invalide pas l'intérêt de la démarche, mais il en précise clairement les limites. Plusieurs facteurs peuvent l'expliquer : la définition très large de ce qu'est une anomalie, le fait que certaines pathologies restent visuellement proches des cas normaux à basse résolution, et la capacité de l'autoencodeur à reconstruire aussi des motifs anormaux simples. Dans le cadre du projet, la brique anomalie doit donc être présentée comme un **signal exploratoire de vigilance** plutôt que comme un détecteur fiable de pathologies.

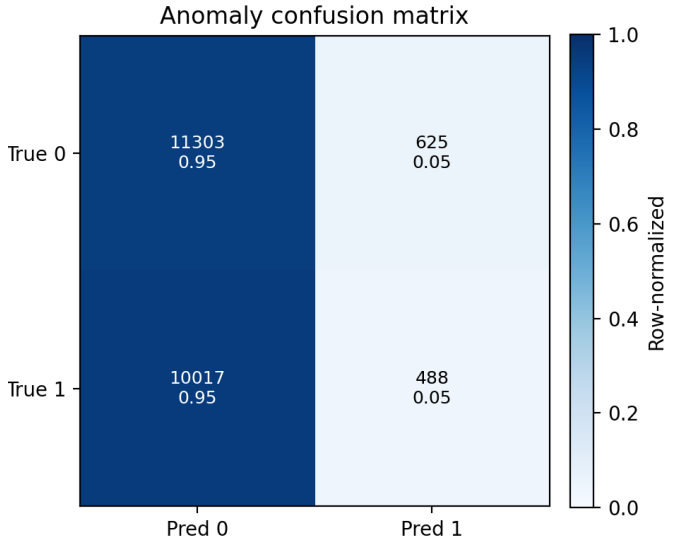

Figure 7 – Matrice de confusion binaire de la brique anomalie au seuil calibré.

9.5 Lecture finale des runs multimodaux

Les résultats multimodaux sont maintenant complets et très instructifs. La branche **image only** obtient un ROC-AUC macro de 0.5663, une AP macro de 0.1463 et un F1 macro de 0.1726. La branche **text only** atteint au contraire un ROC-AUC macro de 0.9953, une AP macro de 0.9823 et un F1 macro de 0.9445. La **fusion image + texte** obtient le meilleur score absolu avec un ROC-AUC macro de 0.9967, une AP macro de 0.9881 et un F1 macro de 0.9583.

La lecture de ces chiffres doit cependant rester très prudente. Le gain de la fusion sur le texte seul existe, mais il reste faible en valeur absolue. Surtout, le test de robustesse montre que lorsque le texte est supprimé, le ROC-AUC macro du modèle de fusion chute à 0.5480, alors que lorsqu'on retire l'image il reste à 0.9966. Autrement dit, dans la configuration actuelle, le modèle de fusion dépend presque entièrement de la modalité textuelle.

Ce comportement est cohérent avec la construction du dataset IU X-Ray dans le projet. Les labels y sont des *weak labels* extraits du texte, ce qui crée un lien direct entre la modalité textuelle et la cible. Le rapport peut donc conclure que la multimodalité est **fonctionnelle techniquement** et que la fusion obtient le meilleur score, mais il doit aussi souligner honnêtement que cette supériorité reflète en grande partie la richesse de la modalité texte et le mode de construction des labels, davantage qu'un apport massif de l'image.

9.6 Snapshot final complet

Le snapshot MLflow intégré au rapport ne contient plus de cellules *n.d.* pour les sept runs finaux attendus. Cette complétude renforce la solidité de la comparaison finale, puisque les conclusions reposent désormais sur des runs réellement terminés, tracés et sauvegardés, et non sur des résultats intermédiaires ou des quickstarts.

9.7 Analyse par classe et implications

L'analyse par classe permet d'identifier plusieurs profils de difficulté. Les classes très rares comme `hernia` souffrent d'abord d'un problème de support statistique. Les classes plus fréquentes mais hétérogènes comme `infiltration` ou `mass` posent un autre type de difficulté : leur phénotype radiologique est moins homogène et donc plus difficile à capturer avec un modèle compact.

Ces profils ont des conséquences pratiques sur le tri. Une classe à bon ROC-AUC mais faible F1 peut encore être utile comme indicateur de priorité si le système est utilisé pour classer les examens par risque plutôt que pour produire une décision dure. En revanche, si l'on souhaite déployer un seuil opérationnel unique, une calibration spécifique par classe devient presque indispensable.

Section 10

Tracking MLflow et traçabilité

La traçabilité est assurée avec MLflow, qui enregistre pour chaque run :

— les hyperparamètres aplatis à partir des fichiers YAML ;
— les métriques par époque et les scores de test ;
— les figures de courbes, histogrammes, reconstructions, barplots et matrices de confusion ;
— le meilleur checkpoint sauvegardé localement et journalisé comme artefact.

Les expériences sont structurées en au moins trois expériences MLflow :

— `chestmnist_supervised` ;
— `chestmnist_anomaly` ;
— `radiology_multimodal`.

La cohérence entre entraînement et déploiement est assurée par le fait que l'application Streamlit recharge directement les checkpoints `best_model.pt`, `best_autoencoder.pt` et `best_multimodal_mo` produits par les scripts d'entraînement. L'application n'utilise donc pas des poids différents de ceux sauvegardés pendant le suivi expérimental.

10.1 Organisation pratique des runs

Chaque famille d'expériences possède son propre espace logique, ses propres configurations YAML et son propre dossier d'artefacts. Cette organisation a plusieurs avantages. Elle simplifie la comparaison entre runs apparentés, elle évite de mélanger les checkpoints, et elle facilite la justification du "meilleur modèle retenu" dans le rapport final.

Du point de vue méthodologique, cette structure fait aussi gagner du temps à l'équipe. Lorsqu'un run produit un comportement inattendu, il est possible de remonter rapidement à la configuration résolue, au type de normalisation, au nombre d'époques effectivement parcourues, à la durée totale du run et aux figures sauvegardées. MLflow ne sert donc pas uniquement à montrer des courbes en fin de projet ; il sert aussi de journal de bord technique pendant tout le développement.

10.2 Artefacts suivis

Table 12 – Principaux artefacts suivis dans MLflow selon la famille de run.

Famille de run	Artefacts enregistrés
Supervision	configuration résolue, checkpoint meilleur modèle, courbes d'entraînement, ROC-AUC par classe, AP par classe, matrices de confusion et métriques test
Anomalie	configuration résolue, meilleur autoencodeur, courbes d'entraînement, histogramme des scores, grille de reconstructions, matrice de confusion et métriques test
Multimodal	configuration résolue, meilleur modèle, courbes d'entraînement, ROC-AUC par classe, matrices de confusion, robustesse à modalité manquante et métriques test

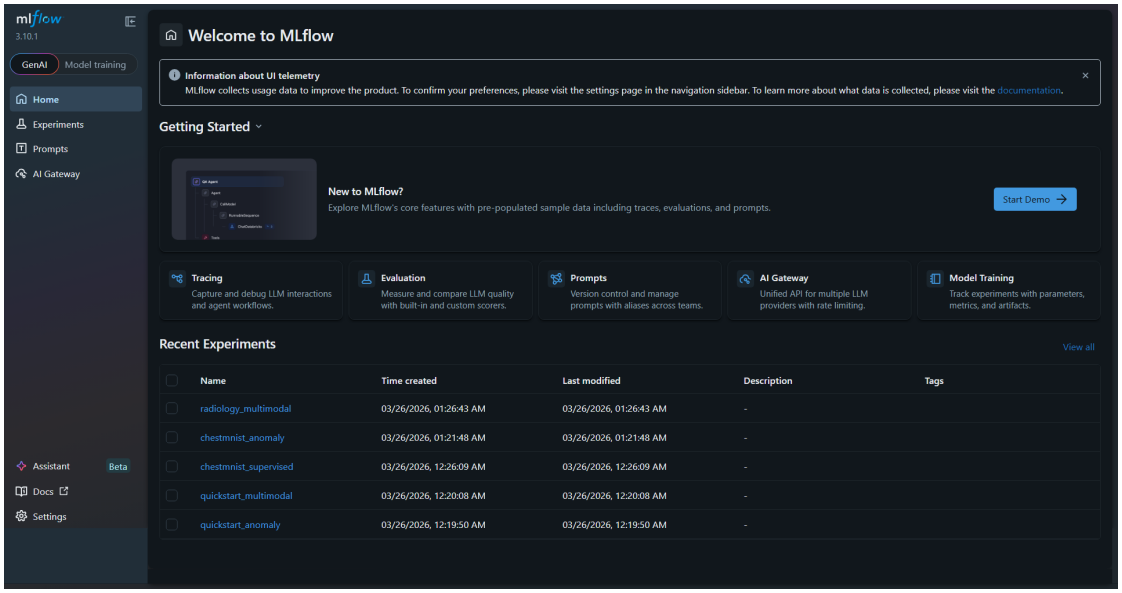

Figure 8 – Capture d'écran de l'interface MLflow locale utilisée pour suivre les expériences du projet. On y retrouve les expériences principales, la navigation entre runs et l'accès aux métriques, paramètres et artefacts.

10.3 Traçabilité jusqu'au déploiement

L'un des points les plus importants du sujet est la cohérence entre le modèle retenu et le modèle effectivement exposé dans l'application. Dans ce projet, cette cohérence est assurée par conception : Streamlit recharge les checkpoints finaux sauvegardés par les scripts d'entraînement, avec leur configuration, leurs labels et, pour la multimodalité, leur vocabulaire. Le démonstrateur pointe désormais par défaut vers **ResNet18 partial FT 224** pour la supervision, **Conv Autoencoder** pour l'anomalie et **Fusion IU X-Ray** pour la multimodalité.

Pour rendre cette correspondance explicite et auditabile, un manifest de déploiement est généré dans `artifacts/deployment_manifest.json`. Il associe chaque checkpoint exposé dans l'application à son expérience MLflow, à son `run_name` et à son `run_id`. Le tableau de traçabilité inséré automatiquement dans le rapport reprend ces informations et constitue la preuve que les

modèles affichés dans la démo correspondent bien à des runs tracés.

Cette continuité est précieuse d'un point de vue académique. Elle permet de démontrer que le démonstrateur n'est pas une interface décorrélée du reste du travail, mais bien l'extension applicative du pipeline expérimental.

Section 11

Démonstrateur applicatif

Le prototype applicatif est implémenté avec Streamlit [7]. L'utilisateur peut charger une radiographie thoracique, obtenir :

— les prédictions supervisées les plus probables ;
— un score d'anomalie avec seuil et statut textuel ;
— une prédiction multimodale si un texte est fourni.

L'architecture de l'application est volontairement simple : chaque composant charge son checkpoint, son prétraitement et, pour la branche multimodale, son vocabulaire. Cette séparation rend le prototype plus lisible et facilite la maintenance.

Les limites du démonstrateur sont cependant claires :

— absence de serveur de production ;
— absence de gestion fine des erreurs et de l'authentification ;
— visualisation centrée sur la prédiction, sans explication clinique profonde ;
— dépendance aux checkpoints disponibles localement.

11.1 Architecture logicielle de l'application

Le démonstrateur s'appuie sur une logique modulaire. Trois bundles peuvent être chargés séparément :

— un bundle supervisé avec modèle, transformations et noms de labels ;
— un bundle anomalie avec autoencodeur, transformations et seuil ;
— un bundle multimodal avec modèle, vocabulaire, transformations et labels.

Cette architecture présente deux avantages majeurs. D'une part, chaque brique peut être remplacée sans casser les autres. D'autre part, l'application peut fonctionner même si tous les runs finaux ne sont pas encore disponibles, ce qui est très utile pendant la phase de développement du projet.

11.2 Scénario d'usage

Du point de vue utilisateur, le parcours est volontairement simple. L'utilisateur charge une image thoracique, consulte les probabilités des pathologies les plus probables, visualise le score d'anomalie et, s'il dispose d'un texte, peut comparer l'effet de la modalité supplémentaire. Ce scénario est cohérent avec un usage de tri ou de démonstration en cours, dans lequel un interne, un enseignant ou un étudiant veut voir rapidement comment plusieurs briques IA réagissent sur le même examen.

Le prototype ne cherche pas à simuler un environnement hospitalier complet. Il s'agit d'une interface de test locale. Ce positionnement est assumé et cohérent avec la nature du projet : l'objectif est de rendre les modèles manipulables et comparables, pas de livrer un logiciel médical certifiable.

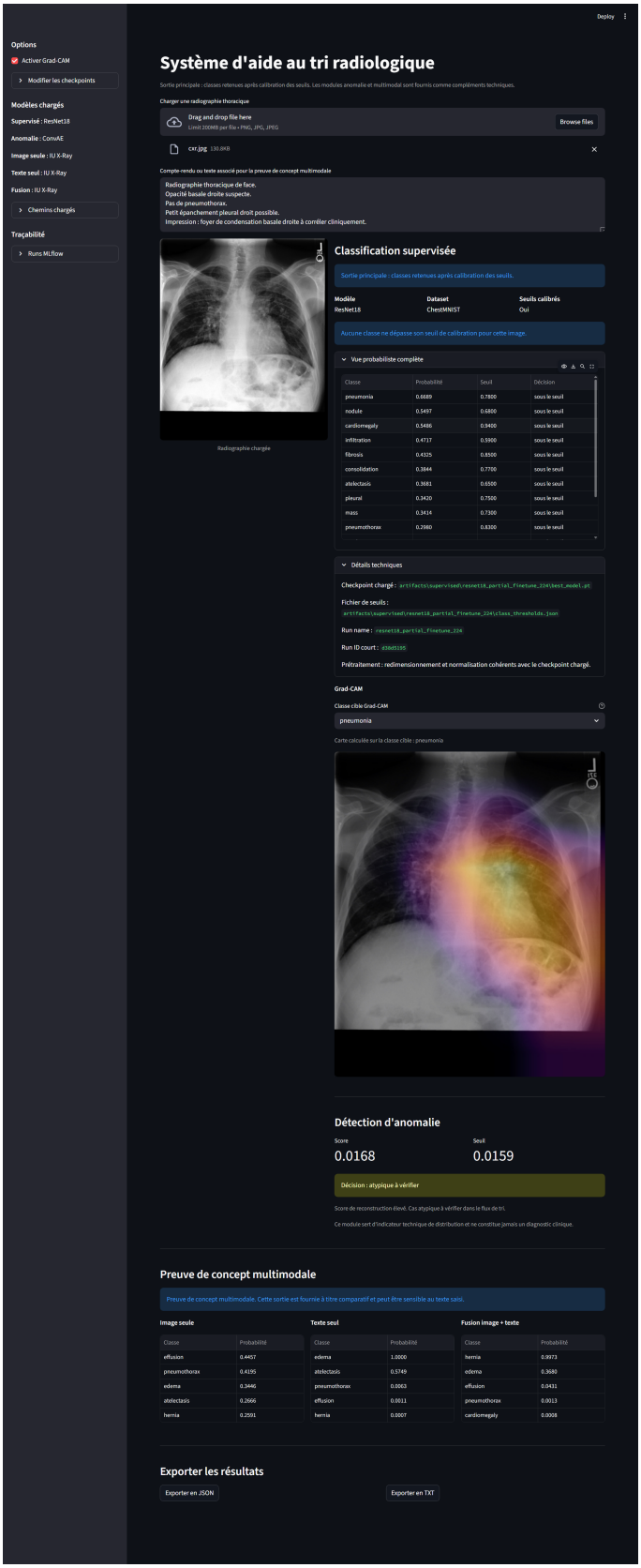

Figure 9 – Capture d'écran du démonstrateur Streamlit final. L'interface rassemble la sortie supervisée principale, le score d'anomalie, une visualisation Grad-CAM optionnelle et le module multimodal utilisé à titre comparatif.

11.3 Limites applicatives et perspectives

Si le projet devait être prolongé au-delà du cadre académique, plusieurs extensions seraient naturelles : ajout d'explications visuelles de type cartes de saillance, historique des prédictions, journalisation des inférences, gestion de plusieurs utilisateurs, validation des formats DICOM et packaging applicatif plus robuste. Ces pistes dépassent le périmètre initial, mais elles montrent que la base actuelle est compatible avec une montée en maturité.

Section 12

Analyse critique

Plusieurs enseignements ressortent du projet.

Premièrement, la partie supervisée montre qu'un **transfer learning adapté** constitue finalement la meilleure base du projet. Le run **ResNet18 partial FT 224** surpasse **SimpleCNN**, **ResNet18 transfer** à backbone gelé et **TinyViT**. Ce résultat suggère qu'un backbone pré-entraîné peut apporter un vrai gain à condition d'autoriser une adaptation partielle au domaine cible. La progression du F1 après calibration rappelle toutefois que le problème n'est pas réduit à un simple classement ; il reste fortement sensible au choix des seuils et au déséquilibre des classes.

Deuxièmement, la détection d'anomalies par reconstruction reste pertinente conceptuellement pour signaler des cas atypiques, mais le run final montre aussi ses limites très clairement. Avec un ROC-AUC test voisin de 0.48, l'autoencodeur ne parvient pas à distinguer de manière fiable les images normales et pathologiques selon la définition retenue. Dans le projet, cette brique doit donc être présentée comme une preuve de concept technique et non comme un module de décision robuste.

Troisièmement, la multimodalité apporte un intérêt fort pour l'analyse radiologique, mais elle doit être maniée avec rigueur. Le fait que **text only** et **fusion** atteignent des scores presque parfaits, tandis que la suppression du texte fait chuter fortement la performance, confirme qu'une part essentielle du signal provient de la modalité textuelle et du mode de construction des weak labels. Le choix d'une fusion intermédiaire et d'un test de robustesse à modalité manquante permet justement de discuter cet aspect au-delà d'un simple score final.

Enfin, le compromis performance / coût de calcul a été central tout au long du projet. Une machine de type portable gamer avec RTX 2070 Max-Q permet de lancer le pipeline final, mais impose des arbitrages : taille d'image modérée, batch size raisonnable, backbone figé en transfer learning, et Vision Transformer compact plutôt qu'un ViT lourd pré-entraîné.

12.1 Lecture d'ensemble des résultats

Le projet met en évidence une tension classique en imagerie médicale : ce qui est facile à bien classer n'est pas toujours ce qui est le plus important cliniquement, et ce qui est important cliniquement est souvent rare, ambigu ou co-occurrent avec d'autres signes. La valeur du pipeline ne vient donc pas d'un seul chiffre de test, mais de la combinaison de plusieurs sorties : probabilités supervisées, score d'anomalie, robustesse multimodale et visualisation dans l'application.

Cette lecture systémique est importante, car elle reflète mieux l'esprit du sujet. Un bon projet ne consiste pas seulement à obtenir le meilleur score sur un benchmark ; il consiste à articuler plusieurs briques complémentaires de manière cohérente et traçable.

12.2 Robustesse et généralisation

La question de la généralisation reste ouverte. Les datasets utilisés sont publics, de provenance limitée et déjà fortement pré-traités dans le cas de ChestMNIST. Il serait donc abusif d'extrapoler les performances à un autre hôpital, à d'autres protocoles d'acquisition ou à d'autres populations

sans expérience supplémentaire.

La branche anomalie apporte ici un intérêt conceptuel : elle peut servir de filet de sécurité lorsque l'image semble sortir du domaine appris. Ce n'est pas une garantie de généralisation, mais c'est un pas vers une meilleure prudence système.

12.3 Biais, limites et précautions d'interprétation

Plusieurs biais doivent être explicitement rappelés dans la version finale du rapport. Le premier est le biais de classe : certaines pathologies rares ont trop peu d'exemples pour produire des estimations très stables. Le deuxième est le biais de source : les datasets utilisés ne représentent pas toute la variété de la radiologie thoracique réelle. Le troisième est le biais de construction des labels multimodaux sur IU X-Ray, puisque ceux-ci sont extraits du texte lui-même.

Ce troisième point est sans doute le plus sensible. Il ne remet pas en cause l'intérêt du pipeline multimodal comme preuve de concept, mais il impose de présenter les résultats avec honnêteté. Si le modèle texte seul s'avère très fort, il faudra expliquer qu'une partie de cette force vient du lien direct entre la modalité textuelle et les weak labels, et non d'une "compréhension clinique" à proprement parler.

12.4 Intérêt pédagogique et scientifique du projet

Malgré ces limites, le projet conserve un intérêt académique fort. Il oblige à traiter des problèmes réels de données, d'architecture, de protocole, de traçabilité, d'interface et d'interprétation. Il montre aussi qu'un projet de deep learning médical n'est pas réduit au choix d'un backbone. La qualité de la préparation des données, de l'évaluation et de la cohérence logicielle compte au moins autant que la sophistication du modèle.

12.5 Enjeux éthiques et usage responsable

Un système d'aide au tri radiologique touche indirectement à des enjeux sensibles : priorité des patients, risque de faux négatifs, surconfiance dans l'IA et difficulté à expliquer certaines sorties. Même dans un cadre de projet, il est important de rappeler qu'aucune prédiction du pipeline ne doit être interprétée comme un avis médical autonome.

Cette prudence est d'ailleurs cohérente avec le design du démonstrateur : l'application affiche des scores et des messages de vigilance, mais ne présente jamais la sortie comme une décision définitive. Une telle posture est indispensable si l'on veut discuter l'usage responsable de l'apprentissage profond en santé.

Section 13

Conclusion et perspectives

Le projet aboutit à une base cohérente de système d'aide au tri radiologique combinant classification supervisée, détection d'anomalies, multimodalité, traçabilité MLflow et démonstrateur Streamlit. Au vu des résultats finaux, la solution la plus raisonnable à recommander à ce stade est le couple :

— **ResNet18 partial FT 224** comme modèle supervisé principal recommandé sur ChestMNIST, avec calibration de seuils par classe ;
— une brique d'anomalie conservée comme module exploratoire de vigilance, mais pas comme signal décisionnel fort ;
— une branche multimodale utilisée comme enrichissement contextuel et comme preuve de concept, avec une lecture prudente des performances à cause des weak labels textuels.

Les perspectives d'amélioration les plus naturelles sont :

— calibrer les seuils par classe pour améliorer le F1 des modèles supervisés ;
— tester un fine-tuning partiel ou complet du backbone de transfer learning, voire un backbone pré-entraîné en imagerie médicale ;
— remplacer le GRU texte par un encodeur biomédical pré-entraîné ;
— reconstruire la partie multimodale sur des labels indépendants du texte ou sur des annotations plus directement validées ;
— approfondir la partie anomalie avec un VAE ou une approche d'énergie ;
— enrichir l'application avec explications visuelles et export de rapport.

En l'état, le projet constitue une base académique solide, reproductible et extensible pour discuter les enjeux d'un système de tri radiologique assisté par apprentissage profond.

Section 14

*

Références

[1] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister et Bingbing Ni. MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10 :41, 2023. DOI : 10.1038/s41597-022-01721-8.

[2] Xiaosong Wang, Yifan Peng, Le Lu, Zhiyong Lu, Mohammadhadi Bagheri et Ronald M. Summers. ChestX-ray8 : Hospital-Scale Chest X-Ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. URL : openaccess.thecvf.com.

[3] Dina Demner-Fushman, Marc D. Kohli, Marc B. Rosenman, Sonya E. Shooshan, Laritza Rodriguez, Sameer Antani, George R. Thoma et Clement J. McDonald. Preparing a collection of radiology examinations for distribution and retrieval. *Journal of the American Medical Informatics Association*, 23(2) :304–310, 2016. DOI : 10.1093/jamia/ocv080.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren et Jian Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. URL : openaccess.thecvf.com.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov et al. An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*, 2021. URL : arxiv.org/abs/2010.11929.

[6] MLflow. MLflow Documentation. URL : mlflow.org/docs/latest/index.html.

[7] Streamlit. Streamlit Documentation. URL : docs.streamlit.io.